Multi-resource Workload Allocation: Repair versus Penalty Constraint Handling in Local Search and Genetic Algorithms.

This report solves a 0-1 multidimensional knapsack problem (MKP) with a random-restart hill climber (HC) and a steady-state genetic algorithm (GA), each of which takes an interchangeable constraint handler. On instance 1 the repair HC reaches the proven optimum in 30/30 runs, while the penalty HCs never do. On instance 2 a controlled 30-run experiment shows that Lamarckian repair beats an adaptive penalty (30/30 versus 0/30 optimal runs, $p = 6.0\times10^{-13}$, $A_{12} = 1.00$), even at equal wall-clock time and despite costing 2.3× more per evaluation.

Problem and instances

Model j has utility p_j and uses r_{ij} units of resource i, whose capacity is b_i. The choice of which models to deploy is the MKP, which is formulated below.

$$max\ \Sigma_j\ p_j\ x_j \quad subject\ to \quad \Sigma_j\ r_ij\ x_j \leq b_i \ \ (i = 1..m), \quad x_j \in \{0,1\}$$

All algorithms use the binary vector x as their genotype. Although most vectors break a capacity, all vectors are valid selections. We used SciPy's MILP solver to solve both instances exactly; the search never sees the optima (Table 1), which are only used to report gaps and success rates.

Table 1. Instance characteristics (smaller tightness = scarcer resource).

Property	Instance 1	Instance 2
n / m	28 / 2	90 / 5
Search space 2^n	2.68×10^8	1.24×10^{27}
Tightness b_i / Σ_j r_{ij}	0.44, 0.50	0.63, 0.33, 0.85, 0.95, 0.91
LP bound / proven optimum	131,637.5 / 130,883	11,194.7 / 11,191
LP dual weights w_i	4.4, 107	0, 2.77, 0, 0, 0

Both instances are tight, so the optimum lies on the feasibility boundary. A dual-based item ordering is very informative in instance 2, since only resource 2 has a non-zero LP dual and the LP bound is within 0.03% of the optimum.

Software Design

The code consists of two files next to the provided loader. The instance, budget tracker, constraint handlers, operators, hill climber, GA and statistics are all stored in mkp.py, while run.py runs all the experiments in parallel and generates the statistics and figures. Constraint techniques can be switched without affecting the search code because the

algorithms only call handler(x, tracker). The DROP phase of the repair is vectorised, so one cumulative sum determines how many items to remove, and the ADD phase only scans items that fit the remaining slack. A tracker charges one evaluation for each candidate sent to a handler, whether or not it is feasible, and records the best feasible utility and wall-clock time every 50 evaluations. Only NumPy, SciPy and Matplotlib are used.

Constraint Handling Strategies

The first strategy is a Lamarckian repair guided by the LP relaxation [1]. Each item is assigned a pseudo-utility $u_j = p_j / \Sigma_i w_i r_{ij}$, where w_i is the LP-relaxation dual price, so u_j measures profit per unit of the resources that actually bind. DROP removes selected items in increasing u_j until the solution is feasible, after which ADD inserts unselected items in decreasing u_j while they fit, and the repaired vector is written back. This means no evaluation is wasted on infeasible points, every solution is maximal, which reduces the search to a much smaller set that contains the optimum, and write-back keeps the population from drifting into infeasible space. DROP removes the items that are least effective with respect to the binding resources, and slack resources do not skew the ordering. The cost is O(*nm*) per evaluation, compared with a single matrix-vector product for a penalty.

The random repair is the same but uses random DROP and ADD orders. It keeps feasibility and maximality but has no problem knowledge, which isolates the value of the LP ordering.

The static and adaptive penalties subtract a weighted measure of the constraint excess from the profit.

$$F(x) = \Sigma_j p_j x_j \; - \; \lambda \rho \, \Sigma_i \max(0, \Sigma_j r_{ij} x_j - b_i), \quad \rho = \max p_j / r_{ij}$$

When λ = 1, one unit of excess costs at least as much as it could be worth [2]. To keep the search close to the boundary, the adaptive version [3] doubles λ when the best individual has been infeasible for three generations and divides it by 1.5 when it has been feasible for three. When λ changes, the population is re-scored from cached values at no evaluation cost. Without this step, stale scores allow an infeasible incumbent to survive, which was a flaw we discovered and fixed.

The death penalty is the simplest baseline for the hill climber, as it rejects every infeasible candidate.

Table 2. Constraint-handling strategies (mkp.py).

Handler	Always feasible?	Problem knowledge	Main risk

Repair (LP)	Yes, and maximal	LP-dual pseudo-utility	Bias; diversity loss (many to one)
Repair (Random)	Yes, and maximal	none	Weaker guidance
Static penalty	no	Scale ρ only	Penalty either too strong or too weak
Adaptive Penalty	no	ρ + feedback	Oscillation; tuning of k and β
Death penalty	Yes (rejects)	none	Cannot cross infeasible regions

Hill climber on instance 1

Design

- Initialisation inserts the items in a random order as long as they fit, giving a solution that is feasible, maximal and identical for all handlers.
- Move is a single bit flip or, with a probability of 0.5, a swap. At a maximal solution no single flip is a feasible improvement, whereas a swap moves the search along the capacity boundary.
- Acceptance uses first improvement and accepts equal-fitness moves to cross plateaus.
- Restart happens after 300 non-improving evaluations, and the best solution over all restarts is returned.
- Protocol uses 10,000 evaluations and 30 runs per handler with shared seeds.

Results

Table 3. Hill climber on instance 1 (optimum 130,883), 30 runs.

Handler	Mean	Worst	Gap %	Optimal	Evals to best (median)	Time (s)
Repair (LP)	130,883	130,883	0.00	30/30	338	1.42
Repair (Random)	130,883	130,883	0.00	30/30	1,170	1.75
Static penalty	121,844	112,686	6.91	0/30	6,942	0.54
Death penalty	120,305	108,927	8.08	0/30	4,564	0.49

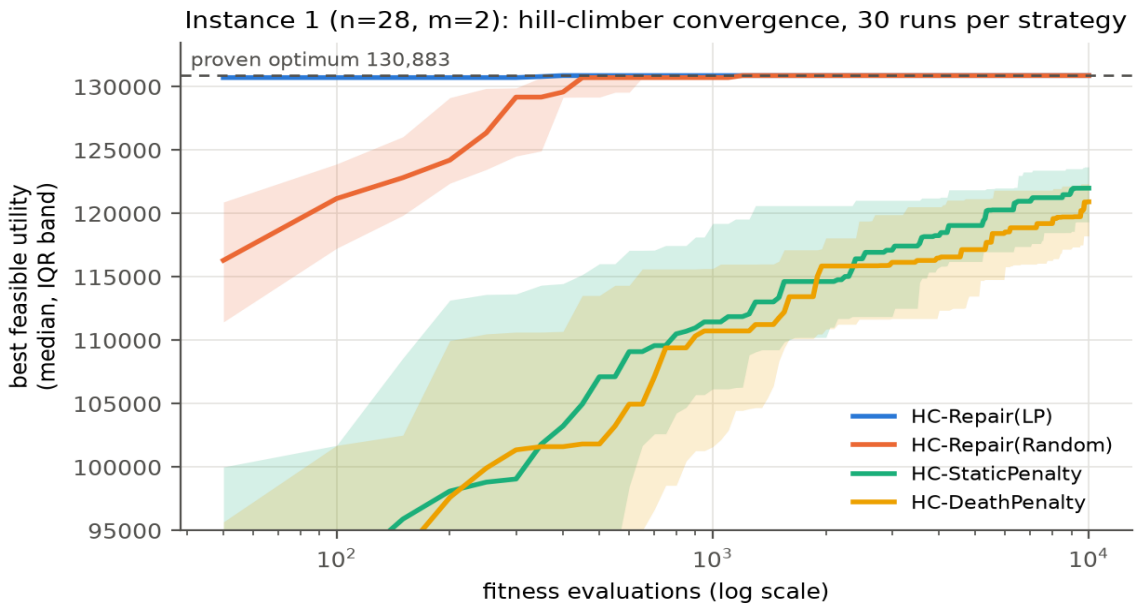


Figure 1. Median best feasible utility with inter-quartile band, 30 runs.

Significant differences exist amongst the handlers (Kruskal-Wallis $p = 4.1\times10^{-22}$). Holm-corrected Mann-Whitney tests reveal that both repairs outperform both penalties ($p = 7.2\times10^{-12}$, $A_{12} = 1.00$), and there is no difference between the death and static penalties ($p = 0.252$). Compared with random ordering, the LP ordering reaches the optimum more quickly (median 338 versus 1,170 evaluations, $p = 2.5\times10^{-5}$).

Because ρ is large, an infeasible neighbour is rarely accepted, so the penalty climbers are limited to feasible flip and swap paths that end in poor local optima (mean 104,085, none of which are optimal). Repair acts as a large compound move that turns each infeasible neighbour into a different maximal solution, and 58% of LP-repair restarts end at the optimum (24% for random repair). For a climber the static penalty therefore behaves almost exactly like the death penalty, which explains why the two are statistically indistinguishable.

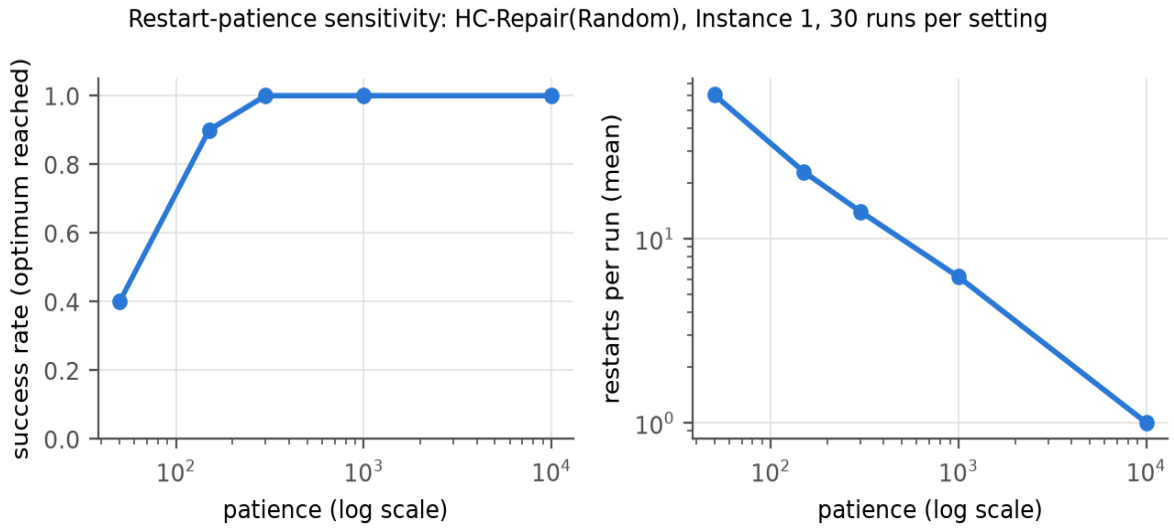


Figure 2. Restart-patience sensitivity (Random-order repair, 30 runs per setting).

Patience below 150 causes climbers to restart before they finish descending (12/30 successes at 50), whereas every run from 300 onwards is successful. Even a single climb that never restarts succeeds every time, which shows how benign the repaired landscape of instance 1 is, but we keep 300 as a safe default for harder instances.

Steady state genetic algorithm

A population can keep many basins open and recombine good partial selections. Following Chu and Beasley [1], the design is listed in Table 4, and all variants share every parameter.

Table 4. GA configuration (ga in mkp.py). One generation = 100 evaluations.

Component	Choice and rationale
Population	100 random feasible maximal solutions, no duplicates; identical across variants.
Selection	Binary tournament, giving mild pressure that is insensitive to fitness scale.
Crossover	Uniform, as item indices have no positional meaning, so the linkage bias of 1-/2-point crossover would be arbitrary.
Mutation	Flip 2 random bits, often an add and remove pair that moves along the boundary.
Replacement	Child replaces the worst member if at least as fit and not a duplicate.

Because the MKP has no ordering among items, a good solution is a set of models whose combined demand satisfies all capacities. Uniform crossover passes on each item decision independently and can therefore combine any subsets of two parents, whereas one- and two-point crossover only exchange contiguous index blocks whose borders depend on the arbitrary order of the input file. Uniform crossover is also disruptive and often overfills the knapsack, but this is harmless once repaired, since the child is projected back onto a maximal feasible solution that keeps the parents' most efficient shared items. Two-bit mutation was chosen over per-bit mutation because one add and one remove is the smallest move that can change a maximal solution without being instantly undone by repair, while a single flip of a maximal solution is either infeasible or strictly worse. Binary tournament keeps selection pressure low to offset repair's tendency to create near-identical offspring, and duplicate elimination enforces a diverse population of unique solutions.

Lamarckian repair was preferred because writing the repaired vector back makes the genotype equal to the evaluated phenotype, so crossover recombines solutions that are known to be feasible and maximal. Baldwinian repair would keep infeasible genotypes

and could preserve more diversity, but parents would then pass on choices that repair systematically reverses, which wastes variation.

Controlled experiment on Instance 2

Hypotheses

The primary hypothesis, H1, is that under the same budget of 20,000 evaluations A (GA + LP repair) reaches a better final utility and requires fewer evaluations than B (GA + adaptive penalty). Approximately 60% of offspring are infeasible, and while B spends those evaluations on unusable points, A transforms each into a good maximal feasible solution, so every evaluation samples the region containing the optimum. B's population may also cross the boundary and remain there when λ is low. The counter-hypothesis is that B might catch up at equal time, because repair costs around twice as much per evaluation and its many-to-one mapping can collapse diversity.

The second hypothesis, H2, is that repair's advantage comes mostly from the LP ordering, which is tested with C (random repair).

The third hypothesis, H3, is that the adaptive penalty beats the static penalty (D), since λ = 1 is very conservative.

Protocol

Each variant has 30 independent runs of 20,000 evaluations. For an equal-time check, B was also run with 80,000 evaluations (B×4, mean 7.3 s, at least as long as A's 4.4 s), and its best-so-far value was read off at A's median run time. Every variant begins from the same population because run *r* of each variant uses the same seed. As the results are not normally distributed, we follow Arcuri and Briand [4] and apply the Mann-Whitney U test (one-sided for H1), the Vargha-Delaney A_{12} effect size [5], Fisher's exact test for success rates and Holm's adjustment [6] for families of tests, with α = 0.05.

Results

Table 5. GA variants on instance 2 (optimum 11,191), 30 runs each.

Variant	Mean	Worst	Optimal	Evals to optimum (median)	Time (s)	µs / eval	Feasible final pop
A: Repair (LP)	11,191.0	11,191	30/30	492	4.44	222	100%
B: Adaptive penalty	11,168.0	11,156	0/30	-	1.95	98	56%
C: Repair (Random)	11,191.0	11,191	30/30	7,958	5.11	255	100%
D: Static penalty	11,161.8	11,143	0/30	-	1.72	86	100%

Bx4 (80k evals)	11,189.0	11,187	15/30	49,066	7.30	91	40%

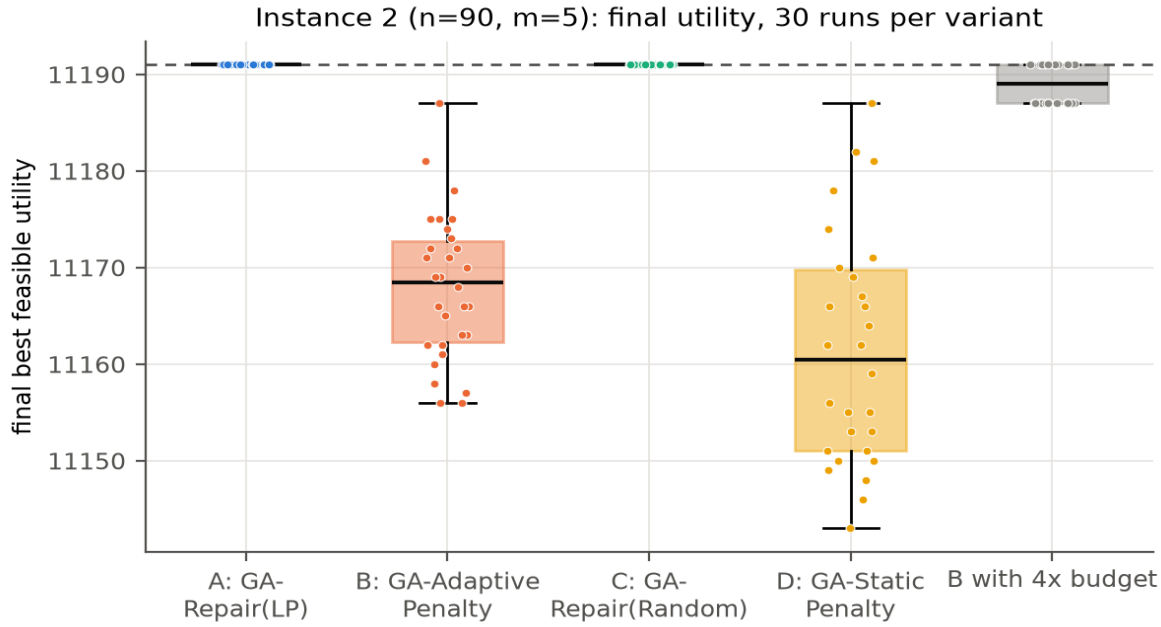


Figure 3. Final utility per run (dashed line = optimum)

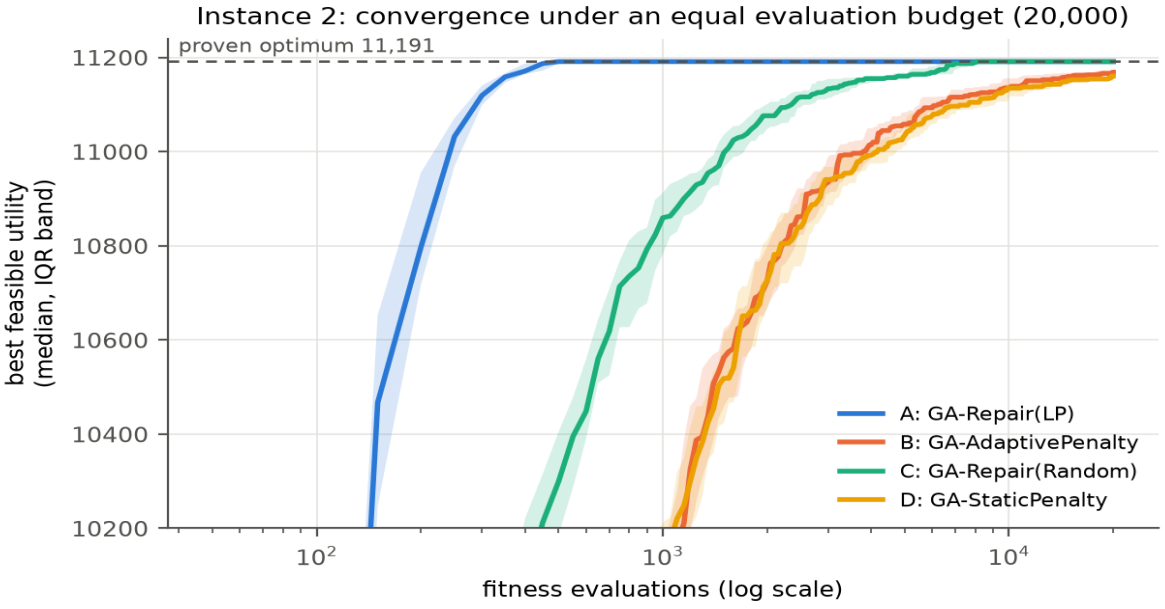


Figure 4. Convergence at equal evaluations (median and IQR, log x-axis)

H1 is supported, as A beats B in every pair of runs ($U = 900$, $p = 6.0\times10^{-13}$, $A_{12} = 1.00$) with 30/30 successes against 0/30 (Fisher $p = 1.7\times10^{-17}$), and it stays ahead at every budget tested from 250 to 20,000 evaluations (Table 6).

Table 6. Anytime comparison at fixed budgets (Mann-Whitney, Holm-corrected).

Evaluations	A median (optimal runs)	B	C	A vs B p_{Holm}	A vs C p_{Holm}
250	11,032 (0)	8,369 (0)	9,532 (0)	3.7×10^{-11}	9.1×10^{-11}
500	11,191 (16)	9,208 (0)	10,300 (0)	3.7×10^{-11}	8.0×10^{-11}
1,000	11,191 (30)	10,057 (0)	10,860 (0)	8.4×10^{-12}	8.5×10^{-12}
2,000	11,191 (30)	10,724 (0)	11,076 (0)	8.4×10^{-12}	8.5×10^{-12}
5,000	11,191 (30)	11,058 (0)	11,160 (2)	8.4×10^{-12}	8.0×10^{-11}
10,000	11,191 (30)	11,135 (0)	11,191 (24)	8.4×10^{-12}	0.022
20,000	11,191 (30)	11,168 (0)	11,191 (30)	8.4×10^{-12}	1.000

The counterhypothesis is rejected, as although A takes 222 µs per evaluation against 98 µs for B, B has reached the optimum in only 7/30 runs by A's median run time of 4.4 s and is still worse than A ($p = 1.9\times10^{-9}$, $A_{12} = 0.88$). Even its full 80,000 evaluations only give 15/30, while A typically reaches the optimum after just 0.17 s (Figure 5).

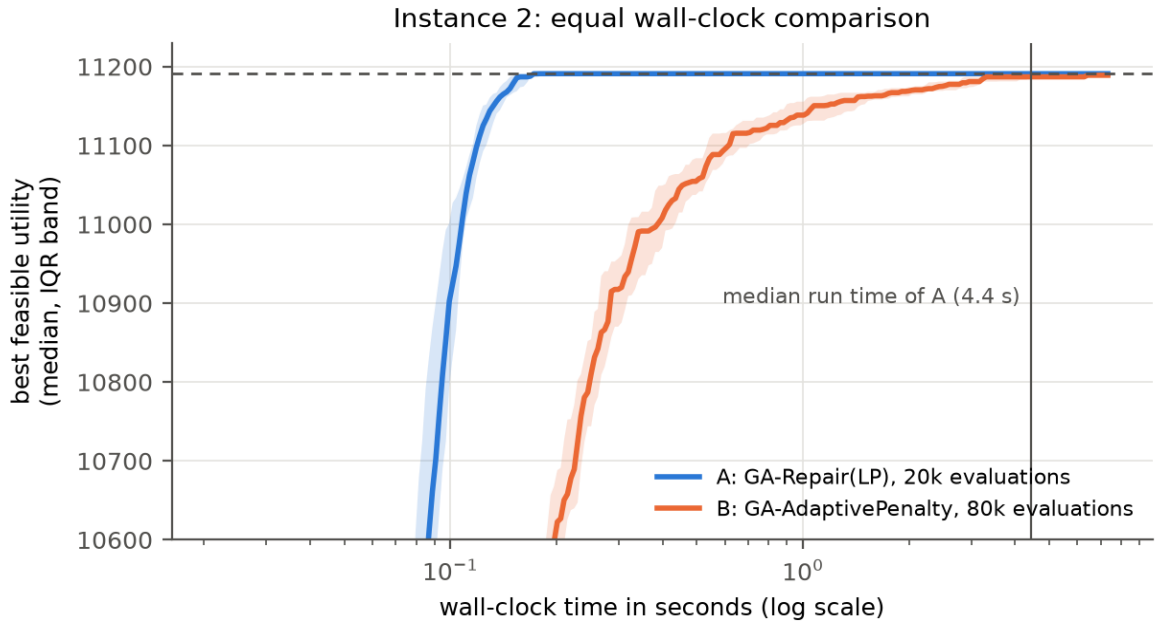


Figure 5. Equal wall-clock comparison (median and IQR, log time axis).

H2 is supported, since random repair also reaches the optimum in 30/30 runs, which shows that feasibility and maximality alone already outperform both penalties at every budget. However, it needs a median of 7,958 evaluations against 492 for A ($p = 1.5\times10^{-11}$), and A is ahead at every budget up to 10,000 (Table 6). Random repair is also the slowest per evaluation, as it draws two permutations every call and drops 2.5 items per evaluation against 1.1, most likely because unguided DROP often removes items that use little of the violated resource.

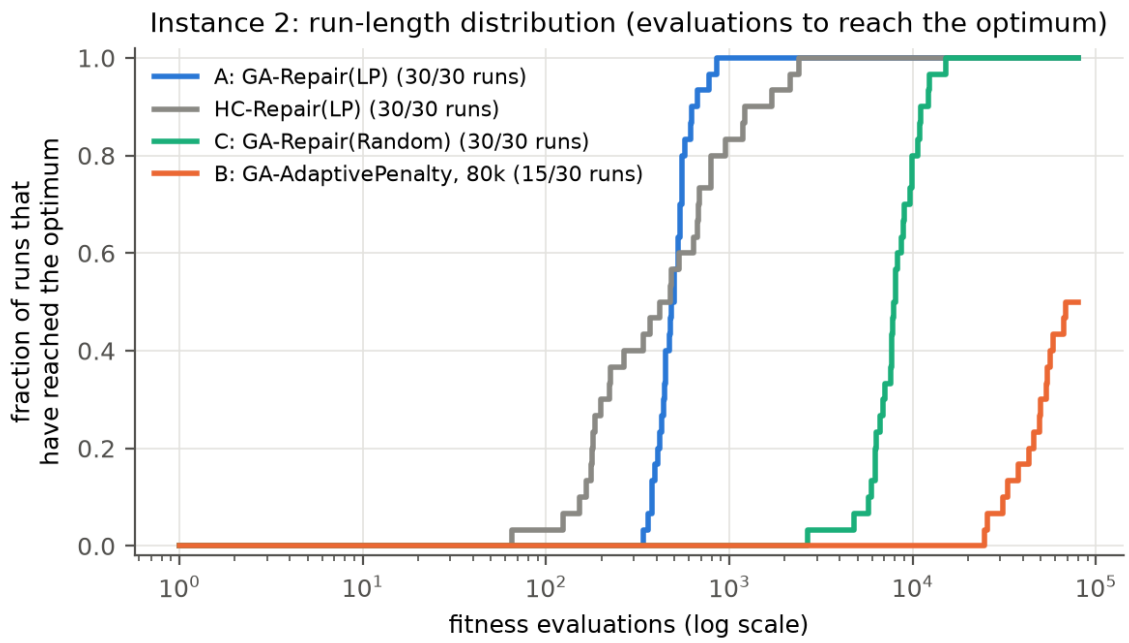


Figure 6. Run-length distributions showing the share of runs that have reached the optimum.

H3 is supported with a modest effect, as the adaptive penalty beats the static penalty (11,168.0 versus 11,161.8, Holm p = 0.034, A_{12} = 0.68). The multiplier λ drops to roughly 0.02 and oscillates there (Figure 7), so the penalty settles close to the real marginal value of the binding resource. Almost half of B's final population is infeasible, meaning it explores both sides of the boundary, while D's population remains 100% feasible.

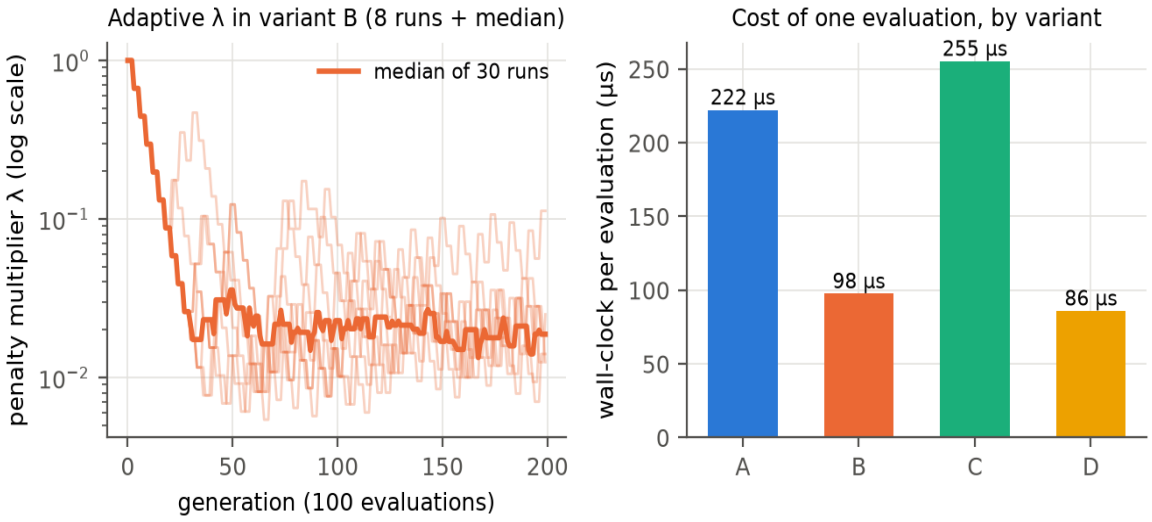


Figure 7. Left, adaptive λ in B. Right, wall-clock cost per evaluation.

With LP repair the HC also solves instance 2 in 30/30 runs, needing a median of 447 evaluations to reach the optimum compared with 492 for the GA (Mann-Whitney p = 0.50, A_{12} = 0.55). Once repair is applied, the population therefore adds no measurable

benefit on this instance, because a single greedy LP-repair pass from an empty vector already gives 11,167, within 0.2% of the optimum, so the remaining search is brief whichever technique is used. Without repair the population does matter, as the static-penalty GA beats the static-penalty HC (11,161.8 versus 11,025.8, $p = 1.3\times10^{-10}$).

Discussion of quality, constraint enforcement and time

Repair dominates the penalties on both instances with maximal effect sizes.

Every variant generates approximately 60% infeasible offspring. Penalty methods spend those evaluations on solutions they are unable to report, whereas repair converts each into a maximal feasible one. As a result, A needs roughly 100× fewer evaluations than B×4 to reach the optimum. Unlike repair, which projects onto the boundary in a single step, the adaptive penalty has to find the boundary through trial and error.

In terms of time, repair makes each evaluation 2.3× slower than a penalty's single matrix-vector product even when vectorised, but its quicker convergence more than makes up for it. Penalties would only be preferable if evaluations were far more expensive than repair or if no repair heuristic existed, and even random repair beat them here. A compiled repair, for example with Numba, would reduce the overhead further without changing the ranking.

Repair is many-to-one, and A generated 12,528 duplicate offspring out of 20,000 (B generated 3). This wastes most of the post-convergence budget and leaves a less varied final population (mean Hamming distance to the best of 4.4 versus 5.7). In harder cases this could lead to premature convergence, for which natural remedies include stagnation-triggered restarts, adaptive mutation or Baldwinian repair.

There are several threats to validity. Instance 2 is easy for repair-based search, since both A and C reach the optimum in every run and the LP ordering alone is almost optimal. Because of this ceiling effect the instance cannot separate the contributions of recombination and repair, which is why A and C are compared on evaluations to the optimum and anytime utility rather than final utility. Harder benchmarks, such as OR-Library MKP instances with n = 250-500 and several binding constraints, are needed to measure what the population adds. In addition, the findings are per-instance and the parameters were taken from the literature without tuning. Times were measured with eight concurrent processes on a laptop, equally for all variants, so only relative times are meaningful.

Conclusion

Repair-based search found the optimum in every run on both instances. On instance 2, LP-guided Lamarckian repair outperformed an adaptive penalty in final quality, success rate and anytime performance, even at equal wall-clock time. The ablation shows that both feasibility and LP knowledge contribute, with LP knowledge cutting the evaluations

needed to reach the optimum by about 16×. On instance 2 an equally repaired hill climber matched the GA, so the population's benefit is yet to be demonstrated on a harder instance. Future work will include diversity-preserving restarts and harder benchmarks with multiple binding constraints.

References

[1] P. C. Chu and J. E. Beasley, "A genetic algorithm for the multidimensional knapsack problem," *J. Heuristics*, vol. 4, no. 1, pp. 63–86, 1998.

[2] A. L. Olsen, "Penalty functions and the knapsack problem," in *Proc. 1st IEEE Conf. Evol. Comput.*, Orlando, FL, USA, 1994, pp. 554–558.

[3] A. B. Hadj-Alouane and J. C. Bean, "A genetic algorithm for the multiple-choice integer program," *Oper. Res.*, vol. 45, no. 1, pp. 92–101, 1997.

[4] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proc. 33rd Int. Conf. Softw. Eng. (ICSE)*, Honolulu, HI, USA, 2011, pp. 1–10.

[5] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *J. Educ. Behav. Statist.*, vol. 25, no. 2, pp. 101–132, 2000.

[6] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.